Introduction to Sorting and Sorting Properties

Data Structures Using C | Sorting Series – Lecture 1

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Ascending vs Descending

Ascending

Smallest to largest:

$0 \leq 1 \leq 9 \leq 11 \leq 21 \leq 22 \leq 34$

Descending

Largest to smallest:

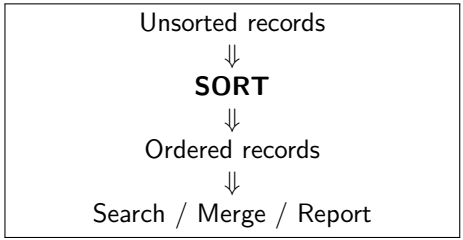
$34 \geq 22 \geq 21 \geq 11 \geq 9 \geq 1 \geq 0$

Important

Use $\leq$ or $\geq$ when duplicate keys are permitted.

Why Sort Data?

- Organize records for presentation.
- Support efficient searching in sorted arrays.
- Facilitate merging of ordered data.
- Group records with similar keys.



Sorting Data in C

Source example

`int A[] = {21,34,11,9,1,0,22};` **Sorted**

ascending

`{0,1,9,11,21,22,34}`

Index	0	1	2	3	4	5	6
Value	0	1	9	11	21	22	34

Index

positions are unchanged; values are reordered.

Two Categories of Sorting

Internal Sorting

Data being sorted fits in main memory.

RAM: Array / Records

External Sorting

Data is too large to fit entirely in memory and is processed using files/storage.

Files $\leftrightarrow$ RAM

Class discussion: Where would sorting a very large collection of log files fit?

Records and Sort Keys

A **sort key** is a field used to order records.

- Name: alphabetical
- Salary: numeric
- Department: categorical/lexicographic

Name	Dept.	Salary
Raj	CSE	890000
Divya	CSE	750000
Aditya	ECE	900000

Sorting on Multiple Keys

Primary key: Department

Secondary key: Name

First arrange departments; within each department arrange names alphabetically.

Name	Department
Divya	Computer Science
Raj	Computer Science
Aditya	Electronics
Huma	Telecommunications
Janak	Telecommunications

Multiple-Key Sorting: Step by Step

Unordered records

Janak	Telecommunications
Raj	Computer Science
Aditya	Electronics
Huma	Telecommunications
Divya	Computer Science

$\Downarrow$ *Department first; name second*

Divya, Raj (Computer Science) $\rightarrow$ **Aditya** (Electronics) $\rightarrow$ **Huma, Janak** (Telecommunications)

Applications of Multiple Sort Keys

Application	Possible key sequence
Employee directory	Department $\rightarrow$ Name
Telephone directory	Location $\rightarrow$ Category $\rightarrow$ Name
Library catalogue	Title $\rightarrow$ Author
Customer addresses	City $\rightarrow$ Street

Question

If two students have the same marks, which secondary key could be used to break the tie?

Practical Considerations for Sorting

1. Key comparisons
2. Record movements/swaps
3. Best-case performance
4. Worst-case performance
5. Average-case performance
6. Stability

Evaluate an algorithm

Comparisons
+
Data movements
+
Input arrangement
+
Stability

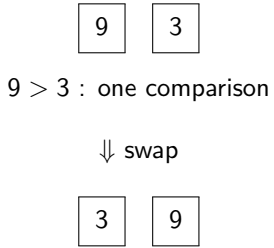
Key Comparisons and Data Movements

Comparison

Compare $A[i]$ and $A[j]$ to decide order.**Movement**

Swap or shift a value/record.

Note: Comparisons and movements are different costs.



Best, Average, and Worst Cases

Case	Meaning
Best	Input arrangement requiring least work
Average	Expected work over a specified input distribution
Worst	Input arrangement requiring most work

Exampl

of inputs:

Already sorted:

1	2	3	4

Reverse sorted:

4	3	2	1

Stability of Sorting

A stable sorting algorithm preserves the relative order of records with **equal sort keys**.If two records both

have marks 80, the one appearing first originally remains first after a stable sort by marks.

Before

80:A	60:C	80:B

Stable result

60:C	80:A	80:B

Why Stability Matters for Multiple Keys

Suppose records are first sorted by **name**. A later **stable** sort by department preserves name order within each department.This is one way to

implement multi-key ordering.

Name	Dept.
Aditya	ECE
Divya	CSE
Huma	Telecom
Janak	Telecom
Raj	CSE

$\Downarrow$ stable department sort

Divya, Raj; Aditya; Huma, Janak

Array vs Linked List: Implementation Context

Property	Array	Linked list
Access	Random access	Sequential access
Storage	Contiguous logical array	Nodes linked by pointers
Sorting implementation	Index comparisons/swaps	Pointer traversal or relinking

The goal of sorting remains the same; the implementation details differ.

Preview: Algorithms Ahead

Method	Central idea
Bubble sort	Repeated adjacent comparisons and swaps
Insertion sort	Insert a key into a sorted prefix
Selection sort	Select minimum of unsorted suffix
Merge sort	Divide, recursively sort, merge
Quick sort	Partition around a pivot

Sort.

Next lecture: Bubble Sort and Insertion

Classroom Activity

Task 1: Sort $\{21, 34, 11, 9, 1, 0, 22\}$ in descending order.

Task 2: Choose primary and secondary sort keys for a university student list.

Task 3: Explain whether the following result is stable by score:

Original: $(80, A), (70, C), (80, B)$
Result: $(70, C), (80, B), (80, A)$

Check Your Understanding

1. Which type of sorting is used when all data fits in RAM?
2. What is a secondary sort key?
3. Does sorting by ascending order forbid duplicates?
4. What does stability preserve?
5. Why measure record movements separately from comparisons?

Answers and Recap

1. Internal sorting.
2. A tie-breaking key applied after the primary key.
3. No; equal keys are permitted.
4. Relative order of records with equal keys.
5. Moving large records can be costly even when comparison count is low.

Takeaway: Sorting is about both *correct ordering* and *performance properties*.

References

- Reema Thareja, *Data Structures Using C*, sections on introduction to sorting, multiple keys and practical considerations.
- User-provided textbook extract for this lecture.

Lecture Feedback

Please share your feedback on concepts, diagrams, examples and pacing.



Thank You!

Next: Bubble Sort and Insertion Sort